

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—The abstract goes here.

Index Terms—IEEE, IEEEtran, journal, LATEX, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE journal papers produced under LATEX using IEEEtran.cls version 1.8b and later.

II. RELATED WORK

III. PRELIMINARIES: LOGIC-GEOMETRIC PROGRAMMING

Logic-Geometric Programming (LGP) provides a unified optimization framework for solving long-horizon manipulation tasks by coupling discrete reasoning with continuous trajectory optimization [1].

A. Mathematical Formulation of LGP

According to the seminal formulation in [1], LGP is defined as a constrained optimization problem over the continuous trajectory x , the discrete action sequence $a_{1:K}$, the logical states $s_{1:K}$, and the switching times $t_{1:K}$:

$$\min_{x, a_{1:K}, s_{1:K}, t_{1:K}} \int_0^T c(x(t), \dot{x}(t), \ddot{x}(t)) dt + \psi(x(T)) \quad (1a)$$

$$\text{s.t. } \forall_{k=1:K} s_k = \text{succ}(a_k, s_{k-1}) \quad (1b)$$

$$s_K \models \mathbf{g} \quad (1c)$$

$$\forall_{k=1}^K h_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) = 0 \quad (1d)$$

$$\forall_{k=1}^K g_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) \leq 0 \quad (1e)$$

$$\forall_{t \in [0, T]} h_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) = 0 \quad (1f)$$

$$\forall_{t \in [0, T]} g_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) \leq 0 \quad (1g)$$

Where Eq. (1a) represents the path and final costs. Eq. (1b) and (1c) ensure the logical sequence $a_{1:K}$ reaches the goal $\mathbf{g}$. Constraints (1d) and (1e) define the kinematic and physical changes at switching times t_k , while (1f) and (1g) enforce path-wide requirements like collision avoidance.

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see <http://www.michaelshell.org/contact.html>).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

B. KOMO as the Motion Planning Engine

LGP leverages k -order Markov Optimization (KOMO) to solve the underlying motion planning problems. KOMO formulates the trajectory as a large-scale non-linear constrained optimization problem, discretizing the path into multiple time steps. However, since KOMO operates in non-convex geometric spaces, its convergence speed is highly sensitive to the complexity of the constraints. In native LGP, high-level logic uncertainty often forces the solver into frequent, costly backtracking to verify physical feasibility.

C. Search Complexity and Branching Factor

A major challenge in native LGP is the exponential growth of the search space. The state-space complexity is represented as $\mathcal{O}(b^D)$, where b is the branching factor determined by the size of the predicate library P , and D is the depth of the action sequence. In complex assembly scenarios, most predicates in P are irrelevant to the specific sub-task, creating a significant computational bottleneck.

D. Kinematic Topology and Tree Constraints

LGP models the world state through a kinematic tree structure, where state transitions are achieved via *Kinematic Switches* that modify the parent-child relationships. To ensure a stable Jacobian-based solver, standard LGP formulations assume each object has exactly one parent frame. This tree-based assumption fails in multi-support assembly tasks where objects induce closed kinematic loops, disrupting standard solver computations.

IV. METHOD

We propose **V-LGP** (Vision-Language Geometric Programming), a hierarchical framework that bridges high-level semantic reasoning with low-level geometric motion planning.

A. Phase 0: Spatio-Geometric Grounding

The pipeline is initialized with a top-down orthographic camera view and an anonymous simulation kinematic graph (`.g` file). V-LGP utilizes a **Geometry-Distance Binding** protocol to ensure Sim-to-Real robustness.

The **VLM Matcher** identifies object candidates and sorts them according to their **Radial Euclidean Distance** from the robot base, autonomously binding raw visual features to semantic simulation IDs.

B. Phase 1: Zero-Shot Topology Extraction

The **VLM Analyst** extracts a semantic **Layered Precedence Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from visual assembly manuals. Our approach utilizes the model’s inherent spatial common sense for direct visual reasoning and deterministic sandboxing.

C. Phase 2: Dynamic Rule Allocation & Batch Compilation

This phase introduces **Dynamic Rule Allocation**. V-LGP prunes the search tree by isolating only the predicates P_{batch} required for the current node. We quantify the complexity reduction as:

$$\text{Complexity}_{\text{V-LGP}} = \sum_{i=1}^N \text{solve}(\text{Node}_i, P_{batch}) \approx N \cdot \mathcal{O}(1) \quad (2)$$

where $P_{batch} \ll P$. This reduces KOMO execution time from over ten minutes to a deterministic two-minute window.

D. Phase 3: Constrained Geometric Execution

The geometric execution phase maps logic to continuous trajectory optimization.

1) *Topological Decoupling via Virtual Anchors*: To handle multi-support scenarios, we introduce **Virtual Anchors** (A_v) via a 6-DOF free joint (`JT_free`). The solver optimizes the state x_t by forcing the anchor to track the geometric centroid of support frames S_i :

$$\min_{x_t} \left(\lambda_1 \sum_{t'=t-1}^t \|\phi_{\text{pose}}(A_v, x_{t'}) - \mathbf{p}_{\text{target}}\|^2 + \lambda_2 \|\phi_{\text{poseDiff}}(A_v, O, x_t)\|^2 \right) \quad (3)$$

2) *Trajectory Shaping via Geometric Funnels*: To prevent lateral collisions, we enforce **Pre-Action Funnels**. For an insertion at time t , we inject:

$$\Phi_{\text{funnel}}(x_{t-\delta}) = \left\| \phi_{\text{posRel}}(O, A_v, x_{t-\delta}) - \begin{bmatrix} 0 & 0 & d_z \end{bmatrix}^T \right\|^2 = 0 \quad (4)$$

where $d_z = 0.05$ m, mimicking human-like precision.

3) *Temporal Collision Relaxation*: We implement a phased collision protocol. During aerial transit ($t \in [t-0.95, t-0.8]$), a safety margin $d_{\text{safe}} = 0.01$ m is enforced. During final placement, this margin is hierarchically relaxed to allow physical contact.

E. Phase 4: State Verification & Structural Checkpoints

The **VLM Judicator** verifies the scene against the expected JSON state. Deviations trigger a `Safe Halt`, establishing a foundation for future error recovery work.

V. EXPERIMENTAL RESULTS

A. Performance Evaluation

Experimental data and baseline comparisons with native LGP go here.

VI. CONCLUSION

The conclusion summarizes the contributions of V-LGP in long-horizon assembly tasks.

APPENDIX A

PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] M. Toussaint, “Logic-geometric programming for multi-step manipulation,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
- [2] H. Kopka and P. W. Daly, *A Guide to LATEX*, 3rd ed., 1999.